

23. Struktura překladače a charakteristika fází překladačnického procesu (lexikální analýza, deterministická syntaktická analýza a generování kódu)

23. Struktura překladače a fáze překladačnického procesu

SZZ okruh 23 (FIT BUT). Od zdrojového kódu k cílovému programu přes posloupnost fází.

Shrnutí

Fáze překladačnického procesu

1. **Lexikální analýza** (scanner), 2. **syntaktická** (parser), 3. **sémantická**, 4. **generování vnitřního kódu**, 5. **optimalizace**, 6. **generování cílového kódu**.
- Frontend (závisí na zdrojovém jazyce) vs. backend (závisí na cílovém); vnitřní kód je rozhraní mezi nimi.

Lexikální a syntaktická analýza

- **LA**: text → **tokeny** (lexémy klasifikované, s atributy); založena na **regulárních výrazech** a **DKA** ([[okruhy/21-regularni-jazyky]]); komunikuje s **tabulkou symbolů**.
- **SA**: tokeny → simulace **derivačního stromu**; deterministické **zásobníkové automaty** ([[okruhy/22-bezkontextove-jazyky]]), podmnožiny BKG: **LL** a **LR**.
 - **Shora dolů (LL)** — First/Follow/Predict, **rekurzivní sestup** nebo prediktivní analýza s LL tabulkou; prázdná buňka = syntaktická chyba.
 - **Zdola nahoru** — **precedenční** (výrazy) a **LR** analyzátory (silnější než LL).

Sémantika a generování kódu

- **Sémantická analýza**: kontrola typů, deklarací, dělení 0; výstup **abstraktní syntaktický strom (AST)**.
- **Vnitřní kód**: většinou **tříadresný (3AK)** — jednotný, snadno se optimalizuje.
- **Optimalizace**: lokální (šíření konstant/kopíí, invarianty cyklu) i globální (mrtvý kód); nepovinná, výsledek funkčně stejný.
- **Cílový kód**: slepé vs. kontextové generování (živé/mrtvé proměnné, registry).

Související syntéza

- [[synthesis/konecne-automaty-napric-obory | Konečné automaty napříč obory]] — syntéza
- [[synthesis/zasobnik-napric-obory | Zásobník napříč obory]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Fáze překladače □ [[#Fáze překladače]] - *Jaké jsou fáze, vstup/výstup?* □ LA (text → tokeny), SA (tokeny → derivační strom), sémantika (→ AST), generování vnitřního kódu, optimalizace, generování cílového kódu. - *Frontend vs. backend?* □ Frontend závisí na zdrojovém jazyce, backend na cílovém; rozhraní = vnitřní kód.

Lexikální analýza □ [[#Lexikální a syntaktická analýza]] - *Lexém vs. token?* □ Lexém = konkrétní řetězec ve zdroji; token = jeho klasifikace (+atributy). - *Čím se implementuje?* □ Regulární výrazy □ deterministický konečný automat.

Syntaktická analýza □ [[#Lexikální a syntaktická analýza]] - *Shora dolů vs. zdola nahoru?* □ Top-down: LL (rekurzivní sestup / prediktivní), levá derivace; bottom-up: precedenční / LR (silnější), pravá derivace. - *Co je v LL tabulce, prázdná buňka?* □ Neterminál × terminál □ pravidlo; prázdná buňka = syntaktická chyba.

Sémantika a kód □ [[#Sémantika a generování kódu]] - *Výstup sémantické analýzy?* □ Abstraktní syntaktický strom (AST); kontrola typů, deklarací. - *Vnitřní kód?* □ Tříadresný kód (např. $C := A + B$), vhodný pro optimalizaci.

Plné znění (ke studiu)

Překladač

Překladač čte **zdrojový program** překládá jej na **cílový program**.

- **Vstup** - Text zdrojového kódu ve zdrojovém jazyce (obvykle vyšší programovací jazyk - **C++**, **Rust**, **Go**, **C#**, **Java**).
- **Výstup** - Cílový program napsaný v cílovém jazyce (obvykle binární kód nebo bytecode), který je **funkčně ekvivalentní** se zdrojovým programem.

Fáze překladače

[[media/szz-23/media/image11.png]]

Jednotlivé části mohou často být spojené. **Někdy** se provádí **více průchodů** (definice funkce může být až za jejím voláním, optimalizace, ...).

1. **Lexikální analýza:** Lexikální analyzátor - **scanner**,
2. **Syntaktická analýza:** Syntaktický analyzátor - **parser**,
3. **Sémantická analýza:** Sémantický analyzátor,
4. **Generování vnitřního kódu:** Generátor vnitřního kódu,
5. **Optimalizace:** Optimalizátor,
6. **Generování cílového kódu:** Generátor cílového kódu.

Lexikální analýza

[[media/szz-23/media/image6.png]]

- **Vstup** - Text ve zdrojovém jazyce.
- **Výstup** - Řetězec tokenů.
- Provádí **rozpoznávání** a **klasifikaci lexémů**, které reprezentuje pomocí **tokenů**.

- Zdrojový program je rozdělen na **lexémy** – logicky oddělené lexikální jednotky (**identifikátory, čísla, klíčová slova, operátory, ...**).
- Každý lexém je reprezentován jako **token**, který může mít **atributy** (u čísla je to jeho hodnota, u proměnné její název, u řetězce vlastní řetězec (jeho obsah), ...).
- Jednotlivé lexémy jazyků jsou navrženy tak, aby je specifikovaly **regulární výrazy** a bylo je možné přijímat deterministickými konečnými automaty.
 - * Na **DKA** je tak založena **implementace** lexikální analýzy - **scanneru**.
 - * Implementujeme pomocí **switch** statement ve **while** cyklu (může být vnořené).
- Odstraňuje prázdné znaky a komentáře.
- Komunikuje s **tabulkou symbolů**.
 - **Informace o identifikátorech** (jméno, typ, konstantní hodnota, počet a typy parametrů v případě funkce, ...) se uchovávají v tabulce symbolů, která má **zásobníkovou strukturu**, což umožňuje např. definovat globální a lokální proměnné.
- Klíčová slova a identifikátory se rozlišují podle **tabulky klíčových slov**.

[[media/szz-23/media/image9.png]]

Syntaktická analýza

- **Vstup** - Řetězec tokenů.
- **Výstup** - **SIMULACE konstrukce derivačního stromu**.

Syntaktický analyzátor (**parser**) kontroluje, zda **řetězec tokenů** reprezentuje **syntakticky správně** napsaný program. Program **je správný**, pokud je k danému **řetězci tokenů** nalezen **derivační strom**, jinak správný není. Simulace konstrukce **derivačního stromu** je založena na **gramatických pravidlech**. Používají se dva přístupy, a to **shora dolů** a **zdola nahoru**. Pro syntaktickou analýzu se používají **deterministické zásobníkové automaty** (terminály jsou **tokeny**), respektive podmnožiny **bezkontextových gramatik** - **LL gramatiky** a **LR gramatiky** (BKG jsou silnější než LL a LR gramatiky).

- **první L**: čtení zleva doprava,
- **druhé L**: levá derivace (leftmost derivation) - nahrazuje se **nejlevější neterminál**.
- **druhé R**: pravá derivace (rightmost derivation) - nahrazuje se **nejpravější neterminál**.

[[media/szz-23/media/image5.png]]

Syntaktická analýza shora dolů Syntaktická analýza shora dolů je založená na **LL gramatikách** a **LL tabulkách**. LL gramatika **bez ϵ pravidel** je BKG $G = (N, T, P, S)$, pro kterou navíc pro každé t, A platí, že $t \in T$, $A \in N$ a existuje maximálně jedno pravidlo $A \rightarrow X_1X_2...X_n \in P$ takové, že: $t \in \text{First}(X_1X_2...X_n)$.

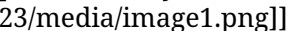
- **First(x)** je množina všech terminálů, kterými může začínat **řetězec derivovatelný z x**, $x \in (N \cup T)^* \sim$ řetězec terminálů a neterminálů.

LL-gramatiky s **ϵ -pravidly** odstraní levé rekurze, ale vyžadují zavést další množiny (kromě **First**): **Empty**, **Follow** a **Predict**. [[media/szz-23/media/image4.png]]

- **First**: Funkce **First(x)** zůstává stejná, ale musíme nyní počítat s ϵ -pravidly. Když terminál t není v množině $\text{First}(B)$, můžeme stále použít pravidlo $A \rightarrow BC\omega$, pokud z B můžeme odvodit ϵ a $t \in \text{First}(C)$. Jestli z neterminálu X můžeme odvodit (derivovat) ϵ , potom $\epsilon \in \text{First}(X)$.

- Empty: **Empty(x)** je množina, která obsahuje jediný prvek ϵ ($\text{Empty}(x) = \{\epsilon\}$), pokud x derivuje ϵ , jinak je prázdná ($\text{Empty}(x) = \emptyset$).
- Follow: **Follow(A)** je množina všech **terminálů**, které se mohou vyskytovat vpravo od A (A je neterminál) ve větě formě.
- Predict: **Predict($A \rightarrow x$)** je množina všech **terminálů**, které mohou být aktuálně nejlevěji vygenerovány, pokud pro libovolnou větnou formu použijeme pravidlo $A \rightarrow x$. 

Implementace LL analyzátoru

- **Rekurzivní sestup**: každý **neterminál** je reprezentován **procedurou/funkcí**, která **řídí** jeho syntaktickou analýzu a může tak **rekurzivně** volat procedury jiných neterminálů dle pravidel. Např. pro **pravidlo** $E \rightarrow TF$ volá procedura pro neterminál E nejdříve proceduru pro neterminál T a poté (pokud je T úspěšná) volá proceduru pro neterminál F a když obě uspějí, vrací také úspěch. Zásobník je zde implicitně dán rekurzí.
- **Prediktivní syntaktická analýza**: Využívá syntaktický analyzátor se **zásobníkem**, který je **řízený LL tabulkou**. **Pravá strana** pravidel (z gramatiky) se ukládá na zásobník **obráceně – reversal** – a provádí se vždy syntaktická analýza neterminálu na **vrcholu** zásobníku. Např. pro každý neterminál může být definováno pole procedur/funkcí, které provádí pravidlo při načtení symbolu. Pokud pro daný symbol procedura není, dochází k chybě. 

Syntaktická analýza zdola nahoru Provádí Pravý rozbor = reverzovaná posloupnost pravidel, která je použita v **nejpravější derivaci** pro **vstupní řetězec**. Analyzátor pracující zdola nahoru dělíme na **precedenční syntaktické analyzátor** (nejslabší, ale jednoduché na implementaci) a **LR syntaktické analyzátor** (nejsilnější, ale složité pro implementaci, jsou silnější než LL analyzátor, protože jdou zdola - mohou existovat „nedeterministická pravidla“).

Precedenční syntaktický analyzátor

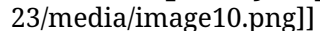
- **Nesmí** existovat více pravidel se **stejnou pravou stranou**.
- Gramatika **nesmí** obsahovat **ϵ -pravidla**.

Pro syntaktickou analýzu využívá **precedenční tabulku**, která je dána **asociativitou** a **precedencí operátorů**. Používá se zejména k syntaktické analýze matematických výrazů (přiřazení do proměnných). V obrázku níže **červené záhlaví** precedenční tabulky představuje symboly na **vstupu** a **žluté záhlaví** symboly na **zásobníku**.

Sémantická analýza



- **Vstup** - Simulace konstrukce derivačního stromu.
- **Výstup** - Abstraktní syntaktický strom.

Kontroluje sémantické aspekty programu, tj. provádí **kontrolu typů** a případně **implicitní konverze** (int na double), kontroluje **deklarace funkcí a proměnných**, kontrola **dělení 0**, kontrola **nepoužitých proměnných**, kontrola **pravdivosti logických výrazů** atd. 

Syntaxí řízený překlad Syntaktický analyzátor (parser) řídí:

- Provádění sémantických akcí a
- Generování abstraktního syntaktického stromu.

Generátor vnitřního kódu

- **Vstup** - Abstraktní syntaktický strom.
- **Výstup** - Vnitřní kód.

Generuje **vnitřní kód** - vnitřní reprezentaci programu (většinou **tříadresný**), ten je **jednotný**, lehce se **překládá** do cílového kódu a lehce se **optimalizuje**. Generování vnitřního kódu může být prováděno rekurzivně na základě abstraktního syntaktického stromu. Syntaktický analyzátor, který pracuje metodou **zdola nahoru**, může generovat tříadresný kód **přímo** bez tvorby ASS. Přímé generování 3AK je založeno na **postfixové notaci**.

Optimalizátor

[[media/szz-23/media/image3.png]]

[[media/szz-23/media/image7.png]]

- **Vstup** - Vnitřní kód.
- **Výstup** - Optimalizovaný vnitřní kód.

Snaží se o optimalizace vnitřního kódu. V rámci **globální** optimalizace odstraňuje mrtvý (nedosažitelný) kód a v rámci **lokální** optimalizace optimalizuje kód v bloku. Lokální optimalizace zahrnují např. **šíření konstanty**, **redukci logických výrazů**, **šíření kopírováním**, **rozbalení cyklu**, **výrazové invarianty v cyklu** (výpočet, který se provádí stejně při každém průchodu cyklem) atd. Je možné optimalizovat na **rychlost** nebo na **velikost** výsledného programu. Překladač ale optimalizátor mít vůbec **nemusí** a výsledek programu bude funkčně **stejný**.

Generátor cílového kódu

- **Vstup** - Optimalizovaný vnitřní kód (případně pouze vnitřní kód).
- **Výstup** - Cílový program.

Převádí vnitřní kód na cílový program, který je zapsán v cílovém jazyce. Obvykle je to **bytecode**, **kód symbolických instrukcí** (nepřesně tzv. **assembler**) nebo **strojový kód**.

Slepé generování Pro každou **instrukci 3AK** existuje procedura, která generuje příslušný cílový kód.

- **výhody**: jednoduché pro implementaci,
- **nevýhody**: instrukce 3AK je **mimo kontext** ostatních a může tak docházet k **přebytečným** načítáním a ukládáním proměnných do/z registrů.

Kontextové generování Udrží si přehled mezi jednotlivými instrukcemi 3AK. Pracuje na principu, že jestliže je **hodnota** proměnné **v registru** a bude „**brzy**“ použita, **ponech** ji v registru. Proměnné se dělí na **živé** (live - budou ještě v bloku použity) a **mrtvé** (dead - již jejich **hodnoty** nebudou použity, proměnné mohou být ale přepsány a poté **dále používány** – jejich hodnota již ale bude odlišná). U živých proměnných se ještě uchovává **řádek následujícího použití**. Pro označení stavů proměnných (živá/mrtvá) se používá **zpětný algoritmus**: instrukce 3AK se čtou od **konce** bloku směrem k jeho **začátku**.

Zdroje

- SZZ okruh 23 — studijní materiály FIT BUT (szz-23.docx). Obrázky: media/szz-23/.

Navigace: [\[\[index | • Přehled okruhů\]\]](#) · [\[\[okruhy/22-bezkontextove-jazyky | 22. Bezkontextové jazyky\]\]](#) · další: [\[\[okruhy/24-numericke-metody | 24. Numerické metody\]\]](#) [\[\[okruhy/25-numericke-metody | 25. Numerické metody\]\]](#)